

OOP资料

前言：某图灵版大佬们的oop错题集：[错题集 - 飞书云文档 \(feishu.cn\)](#)

线上考试

2018-2019 OOP 期末考试	
✕ True-or-False 10 A, Multiple-Choice - 1 10 A Fill-in-Blank 9 @ Fill-in-Blank - P 3	
1-1 catch (type p) acts very much like a parameter in a function. Once the exception is caught, you can access the thrown value from this parameter in the body of a catch block.. (2分)	Author: 张德慧 Organization: 西安邮电大学
☒ T ☐ F	
1-1 Accepted (2 point(s))	
1-2 Insertion << can be used to output all kinds of primitive types, including the pointers. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-2 Accepted (2 point(s))	
1-3 The reason inline functions are introduced into the C++ is to reduce the complexity of space, i.e. to shorten the code. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-3 Accepted (2 point(s))	
1-4 In C++, only existing operators can be overloaded. (2分)	Author: 张德慧 Organization: 西安邮电大学
☐ T ☒ F	
1-4 Accepted (2 point(s))	
1-5 If you are not interested in the contents of an exception object, the catch block parameter may be omitted.. (2分)	Author: 张德慧 Organization: 西安邮电大学
☐ T ☒ F	
1-5 Wrong Answer (0 point(s))	
1-6 It is possible to access any item in a vector directly via its index. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-6 Accepted (2 point(s))	
1-7 Functions with the same name can be identified via namespaces. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-7 Accepted (2 point(s))	
1-8 To make functions overloaded, the parameter list of the functions have to be different from each other. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-8 Wrong Answer (0 point(s))	
1-9 Constructors are able to be declared as virtual. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-9 Accepted (2 point(s))	
1-10 Manipulators are objects to be inserted or extracted into/from streams. (2分)	Author: 翁恺 Organization: 浙江大学
☐ T ☒ F	
1-10 Accepted (2 point(s))	

2018-2019 OOP 期末考试

True-or-False 10

A. Multiple-Choice - 1 10

Fill-in-Blank 9

Fill-in-Blank - P 3

2-1 Given code below:

```
vector<int> v;  
for ( int i=0; i<4; i++ ) {  
    v.push_back(i+1);  
}  
cout << v.size();
```

Author: 翁恺
Organization: 浙江大学

The output should be: (2分)

- ☐ A. 1
- ☐ B. 2
- ☐ C. 3
- ☒ D. 4

2-1 Accepted (2 point(s))

2-2 About virtual function, which statement below is correct? (2分)

- ☐ A. Virtual function is a static member function
- ☐ B. Virtual function is not a member function
- ☒ C. Once defined as virtual, it is still virtual in derived class without virtual keyword.
- ☐ D. Virtual function can not be overloaded.

Author: 翁恺
Organization: 浙江大学

2-2 Accepted (2 point(s))

2-3 It is better to choose ____ when the function is not complicated and is to be called frequently. (2分)

- ☐ A. overloaded function
- ☒ B. inline function
- ☐ C. recursive function
- ☐ D. embedded function

Author: 翁恺
Organization: 浙江大学

2-3 Accepted (2 point(s))

2-4 Suppose that statement3 throws an exception of type Exception3 in the following statement: (2分)

```
try {  
    statement1; statement2; statement3; }  
catch (Exception1 ex1) { }  
catch (Exception2 ex2) { }  
catch (Exception3 ex3) { statement4; throw; }  
statement5;
```

Which statements are executed after statement3 is executed?

- ☐ A. statement2
- ☐ B. statement3
- ☒ C. statement4
- ☐ D. statement5

Author: 张德慧
Organization: 西安邮电大学

2-4 Accepted (2 point(s))

2-5 What is wrong in the following code?

vector v; v[0] = 2.5; (2分)

- ☒ A. The program has a compile error because there are no elements in the vector.
- ☐ B. The program has a compile error because you cannot assign a double value to v[0].
- ☐ C. The program has a runtime error because there are no elements in the vector.
- ☐ D. The program has a runtime error because you cannot assign a double value to v[0].

Author: 张德慧

Organization: 西安邮电大学

2-5 Wrong Answer (0 point(s))

2-6 Given:

```
template <class T>
void max(T a, T b, T &c)
{
    c = a+b;
}
```

Author: 翁恺

Organization: 浙江大学

Which code fragment below is correct? (2分)

- ☐ A. `int x,y;char z;max(x,y,z);`
- ☒ B. `double x,y;double z;max(x,y,z);`
- ☐ C. `int x,y;float z;max(x,y,z);`
- ☐ D. `float x,y;double z;max(x,y,z);`

2-6 Accepted (2 point(s))

2-7 About `const` data member, which statement below is correct? (2分)

- ☐ A. `const` member can be defined without any initialization, and can not be modified.
- ☒ B. `const` member has to be initialized, and can not be modified.
- ☐ C. `const` member can be defined without any initialization, and can be modified later.
- ☐ D. `const` member has to be initialized, and can be modified later.

Author: 翁恺

Organization: 浙江大学

2-7 Accepted (2 point(s))

2-8 Which operator below can not be overloaded? (2分)

- ☐ A. `&&`
- ☐ B. `[]`
- ☒ C. `::`
- ☐ D. `<<`

Author: 翁恺

Organization: 浙江大学

2-8 Accepted (2 point(s))

2-9 Which one below can NOT be overloaded? (2分)

- ☐ A. member function
- ☐ B. free function (global function)
- ☒ C. destructor
- ☐ D. constructor

Author: 翁恺

Organization: 浙江大学

2-9 Accepted (2 point(s))

2-10 About `delete` operator, which statement below is NOT correct? (2分)

- ☐ A. Only pointers as the result of a `new` operation can be used to be `delete`.
- ☐ B. Destructor will be called automatically during the `delete` operation.
- ☒ C. It is safe to `delete` the same pointer multiple times.
- ☐ D. There's only one pair of `[]` followed to `delete` a multi-dimension array.

Author: 翁恺

Organization: 浙江大学

2-10 Accepted (2 point(s))

5-1 The function template printArrayInfo() computes the minimal, maximal and average value of a two dimension array and prints them out, where nrow is number of rows and ncol is the number of columns.

Author: hulanqing
Organization: 浙江大学
Time Limit: 400 ms
Memory Limit: 64 MB

```
#include <iostream>

template<class T> (1分)
void printArrayInfo(T* (1分) array, int nrow, int ncol)
{
    T (1分) max = array[0], min = array[0];
    double avg = 0 (1分);
    for(int i = 0; i < nrow; ++i)
    {
        for(int j = 0; j < ncol; ++j)
        {
            T val (1分) = array[i*ncol+j] (1分);
            if(val<min (1分)) min = val;
            if(val>max (1分)) max = val;
            avg =avg+static_cast<double>(val) (1分);
        }
    }
    avg /= (nrow*ncol (1分));
    std::cout << "min=" << min << std::endl;
    std::cout << "max=" << max << std::endl;
    std::cout << "avg=" << avg << std::endl;
}

int main()
{
    int ai[2][3]={8,10,2},{14,4,6};
    printArrayInfo(ai[0], 2, 3);
    double af[1][5]={3.4f,4.2f,6.6f,2.4f,-0.9f};
    printArrayInfo(af[0], 1, 5);
    return 0;
}
```

5-1 Accepted (10 point(s))

5-2 The class String is a simple C++ encapsulation of the C character arrays.

```
#include <cstring>
#include <iostream>
#include <stdexcept>

class StringIndexError : public std::out_of_range {
private:
    int index;
public:
    StringIndexError(int idx) : std::out_of_range(""), index(idx) {}
    int getIndex() const
    {
        return index;
    }
};

class String {
private:
    char *m_ptr;
public:
    String(const char *ptr)
    {
        m_ptr = new char[strlen(ptr)+1] (1分);
        strcpy(m_ptr, ptr);
    }
    ~String()
    {
        delete[] m_ptr (1分);
    }
    String &operator+=(const String &str)
    {
        char *s = new char[strlen(m_ptr)+strlen(str.m_ptr)+1] (1分);
```

Author: hulanqing
Organization: 浙江大学
Time Limit: 400 ms
Memory Limit: 64 MB

```

        if (m_ptr)
        {
            strcpy(s, m_ptr);
            delete[] m_ptr;
        }
        strcat(s, str.m_ptr); // appends str.m_ptr to s
        m_ptr = s;
        return *this;
    }
    bool operator==(const String &str) const
    {
        return (strcmp(m_ptr, str.m_ptr) == 0);
    }
    char& operator[](int i)
    {
        if (i >= 0 && i < strlen(m_ptr)) return m_ptr[i];
        throw StringIndexError(i);
    }
    friend ostream& operator<<(ostream &, const String &);
};

ostream& operator<<(ostream &out, const String &str)
{
    return out << str.m_ptr;
}

int main()
{
    String s1("Hello "), s2("world!");

    if (s1 == s2)
        std::cout << "S1==S2" << std::endl;
    else
        std::cout << "S1!=S2" << std::endl;

    s1 += s2;
    std::cout << s1 << std::endl;

    try
    {
        int k = 0;
        while (true)
            std::cout << s1[k++];
    }
    catch (const StringIndexError& ex)
    {
        std::cout << "\nString index is out of range: " << ex.getIndex() << std::endl;
    }

    return 0;
}

```

5-2 Accepted (10 point(s))

5-3 The class Queue implements a circular queue data structure.

Author: hulanqing
 Organization: 浙江大学
 Time Limit: 400 ms
 Memory Limit: 64 MB

```
#include <iostream>

template<class T>
class Queue {
private:
    int capacity;           // capacity of the queue
    T* data;                (1分) // dynamically allocated array of doubles
    int front;              // head of the queue
    int rear;               // tail of the queue
public:
    Queue(int maxsize);
    ~Queue();
    bool empty();
    bool full();
    void push(T a);         // append a double value to the tail of queue
    T pop();                // delete the head element of the queue
};

template<class T> Queue<T>::Queue(int maxsize)
{
    capacity = maxsize;
    data = new T[maxsize] (1分);
```

```
    front = rear = 0;
    std::cout << "queue initialized! ";
}

template<class T> Queue<T>::~~Queue()
{
    delete[] data (1分);
    std::cout << "queue destroyed! ";
}

template<class T> bool Queue<T>::empty()
{
    return (front == rear) (1分);
}

template<class T> bool Queue<T>::full()
{
    return (front == ((rear+1)%capacity)) (1分);
}

//The dynamic array data will be a circular Queue
template<class T> void Queue<T>::push(T a)
{
    if (full())
    {
        exit(0);
    }
    else
    {
        data[rear] (1分) = a;
        rear = (rear+1)%capacity (1分);
    }
}
```



```

template<class T> T Queue<T>::pop()
{
    if (empty())
    {
        exit(0);
    }
    T top=data[front] (1分);
    front = (front+1)%capacity (1分);
    return top;
}

int main()
{
    Queue<double> (1分) q(5);
    std::cout << q.empty();

    q.push(1.3);
    q.push(2.3);
    q.push(3.3);
    q.push(4.3);
    std::cout << q.full();

    q.pop();
    q.pop();
    q.pop();
    q.push(5.3);
    q.push(6.3);
    q.push(7.3);
    std::cout << q.full();

    q.pop();
    q.pop();
    q.pop();
    q.pop();
    std::cout << q.empty();
    return 0;
}

```

5-3 Accepted (10 point(s))



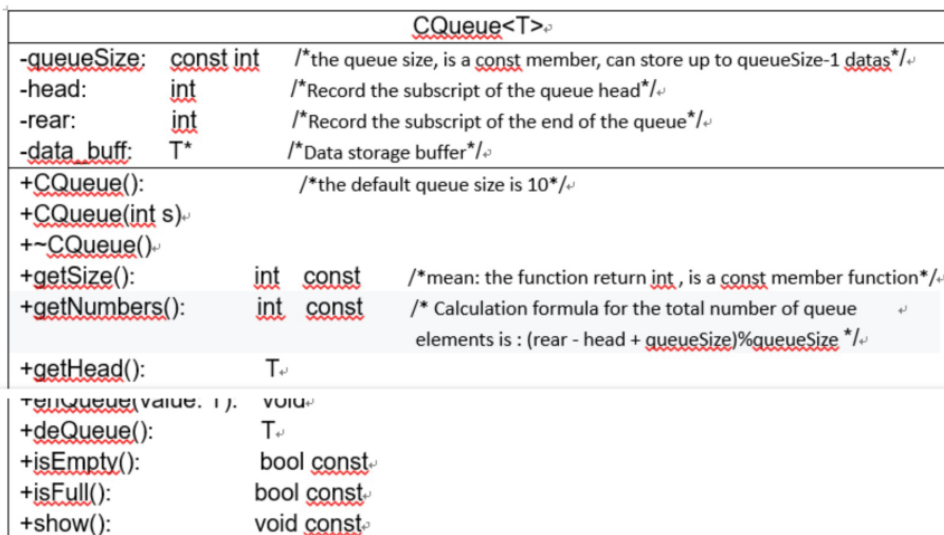
[返回](#)

8-1 Queue (35分)

Design a generic circular queue class. using C++ standard exception class: overflow_error and underflow_error in the design.

the picture below shows the UML class diagram for the generic queue class.

作者 胡兰青
单位 浙江大学



The test code is in test.cpp, The output is (There is a space at the end of the line): now the queue is full! 0 1 2 3 4 5 6 7 8 0 5 6 7 8

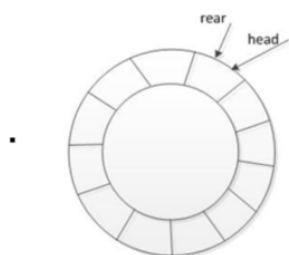


Figure 1 rear==head, The queue is empty

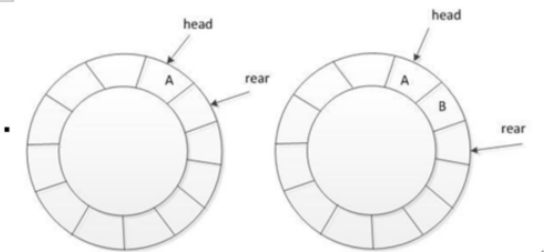


Figure 2 Add elements 'A' and 'B' to the queue in turn

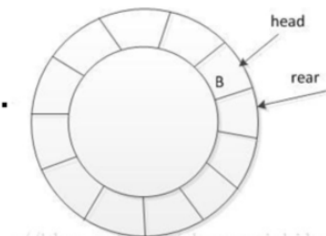


Figure 3 Put the element 'A' of the head out of the queue

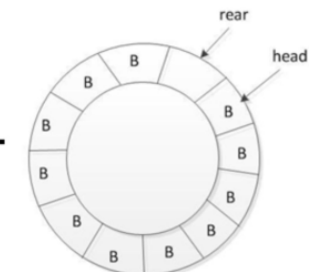


Figure 4 The queue is full when $(rear+1) \% QueueSize == front$

```
/*test.cpp*/
#include <iostream>
#include <stdexcept>
using namespace std;
#include "CQueue.h"
int main()
{
    try {
        CQueue<double> rq;
        for (int i = 0; i < rq.getSize()-1; i++)
            rq.enqueue(i);
        if (rq.isFull()) printf("now the queue is full! ");
        if (!rq.isEmpty()) rq.show();
        cout << rq.getHead() << " ";
        for (int i = 0; i < 5; i++) // dequeuing 5 elements
            rq.dequeue();
        rq.show();
    }
    catch (overflow_error& r)
    {
        cout << r.what();
    }
    catch (underflow_error& r)
    {
        cout << r.what();
    }
    return 0;
}
```

(35分)

公告

这是期末考试的第一部分，在这部分答题时，不能开启考试监考客户端以外的任何软件，包括但不限于浏览器、QQ、微信、支付宝、编程软件等

A 填空题 15

程序填空题 2

5-1

```
#include <iostream>
#include <cstring>
using namespace std;

class Node{
    friend class LinkedList;
    Node *m_pNext;
public:
    Node(){ m_pNext = NULL; }
    virtual ~Node() {}
    void AppendNode(Node &n){n.m_pNext = m_pNext; [1分]; }
    virtual void Print()const = 0;
};

class IntNode : public Node{
    int m_i;
public:
    IntNode(int i){m_i = i;}
    virtual void Print()const{cout << m_i << endl;}
};

class StrNode : public Node
{
    char *m_s;
public:
    StrNode(char *s){ m_s = new char[ [1分] ]; strcpy(m_s,s);}
    ~StrNode(){ [1分]; }
    virtual void Print()const{cout << m_s << endl;}
};

class LinkedList
{
    Node *m_pHead;
    Node *m_pEnd;
public:
    LinkedList(Node &n){ m_pHead = m_pEnd = &n;}
    ~LinkedList(){
        Node *p = m_pHead;
        while (p){ [1分]; delete p; [1分]; }
    }
    void AppendNode(Node &n){
        m_pEnd->AppendNode(n);
        m_pEnd = [1分];
    }
    void PrintList() [1分] {
        Node *p = m_pHead;
        while (p){p->Print(); [1分]; }
    }
};

int main()
{
    char word[80];
    cin >> word;
    LinkedList llist([1分] StrNode(word));
    int i;
    cin >> i;
    llist.AppendNode([1分] IntNode(i));
    llist.PrintList();
}
```

5-1

5-1 ■■■

5-2 要求实现一个表示分数的Fraction类中的部分函数，注意，计算结果不要求约分。

```
class Fraction{
    double numerator;
    double denominator;
public:
    Fraction (double numerator=0.0, double denominator=0.0){
        (1分) ;
        (1分) ;
    }
    Fraction ( (1分) f1){
        numerator = f1. numerator;
        (1分) = f1. denominator;
    }
    Fraction (1分) +(const Fraction &f1);
    (1分) operator- ( (1分) );
    return Fraction((numerator*f1.denominator - numerator*f2.denominator (1分) ), (denominator*f1.denominator));
    (1分) void print(Fraction f);
};
Fraction Fraction::operator+(const Fraction &f1){
    return (1分) ;
}
void print(Fraction f)
{
    cout << "numerator " << f.numerator << endl;
    cout << "denominator " << f.denominator << endl;
}
```

5-2 ■■■

公告

这是期末考试的第一部分，在这部分答题时，不能开启考试监考客户端以外的任何软件，包括但不限于浏览器、QQ、微信、支付宝、编程软件等

填空题 15

程序填空题 2

4-1 The output of the code below is:

```
#include <cstring>
#include <iostream>
using namespace std;

class Str{
    char m_s[10];
    char *m_p;
public:
    Str(char *s){strcpy(m_s,s);m_p = m_s;}
    operator char*(){ return m_p; }
    char *operator++(){ return ++m_p; }
    char operator [](int i){ return m_s[i]; }
};

int main(){
    Str s("ml");
    cout << *s << endl;
    ++s;
    cout << s[0] << endl;
    cout << *s << endl;
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

4-1

4-2 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
public:
    A(){cout << "A()" << endl;}
    ~A(){cout << "~A()" << endl;}
};

class B : public A{
public:
    B(){cout << "B()" << endl;}
    ~B(){cout << "~B()" << endl;}
};

int main(){
    A a;
    B b;
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

The 4th line is (1分)

The 5th line is (1分)

The 6th line is (1分)

4-2

4-3 The output of the code below is:

```
#include <iostream>
using namespace std;

template <typename T>
class FF{
    T a1,a2,a3;
public:
    FF(T b1, T b2, T b3):a1(b1),a2(b2),a3(b3)
    {}
    T Sum() const
    {
        return a1 + a2 + a3;
    }
};

int main()
{
    FF<int> x(2,3,4),y(-2,-3,-4);
    cout << x.Sum() << endl << y.Sum() << endl;
}
```

The 1st line is (1分)

The 2nd line is (1分)

4-3 答案正确 (2分)

4-4 The output of the code below is:

```
#include <iostream>

struct A {
    A() { std::cout << "A" << std::endl; }
    A(const A &a) { std::cout << "B" << std::endl; }
    A& operator=(const A &a) { std::cout << "C" << std::endl; return *this; }
};

int main() {
    A a[2];
    A b = a[0];
    A c;
    c = a[1];
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

The 4th line is (1分)

The 5th line is (1分)

4-4

4-5 The output of the code below is:

```
#include <iostream>

class C {
public:
    explicit C(int) {
        std::cout << "i" << std::endl;
    };
    C(double) {
        std::cout << "d" << std::endl;
    };
};

int main() {
    C c1(7);
    C c2 = 7;
}
```

The 1st line: (1分)

The 2nd line: (1分)

4-5 答案正确 (2分)

4-6 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
public:
    void F(int){ cout << "A::F(int)" << endl; }
    void F(double){ cout << "A::F(double)" << endl; }
    void F2(int){ cout << "A::F2(int)" << endl; }
};

class B : public A{
public:
    void F(double){ cout << "B::F(double)" << endl; }
};

int main(){
    B b;
    b.F(2.0);
    b.F(2);
    b.F2(2);
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

4-6

4-7 The output of the code below is:

```
#include <iostream>

struct A {
    virtual void foo(int a = 1) {
        std::cout << "A" << '\n' << a;
    }
};

struct B : A {
    virtual void foo(int a = 2) {
        std::cout << "B" << '\n' << a;
    }
};

int main () {
    A *a = new B;
    a->foo();
}
```

The 1st line is (1分)

The 2nd line is (1分)

4-7

4-8 The output of the code below is:

```
#include <iostream>
using namespace std;

template<typename T>
T func(T x, double y){
    return x+y;
}

int main(){
    cout << func(2.7,3) << endl;
    cout << func(3,2.7) << endl;
}
```

The 1st line is (2分)

The 2nd line is (1分)

4-8 答案正确 (3分)

4-9 The output of the code below is:

```
#include <iostream>

template<class T> void f(T &i) { std::cout << 1; }

template<> void f(const int &i) { std::cout << 2; }

int main() {
    int i = 24;
    f(i);
}
```

The output is (1分)

4-9 答案正确 (1分)

4-10 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
    int s[10];
public:
    int operator[](int i)const{
        cout << "operator[](int)const" << endl;return s[i];
    }
    int &operator[](int i){
        cout << "operator[](int)" << endl;return s[i];
    }
};

int main(){
    A a1;
    const A &a2 = a1;
    a1[0] = a2[1];
}
```

The 1st line is (1分)

The 2nd line is (1分)

4-10 答案正确 (2分)

4-11 The output of the code below is:

```
#include <iostream>
using namespace std;
class A{
public:
    static void f(double){
        cout << "f(double)" << endl;
    }
    void f(int){
        cout << "f(int)" << endl;
    }
};

int main(){
    const A a;
    a.f(3);
}
```

The output is (1分)

4-12 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
public:
    virtual ~A(){}
};

class B : public A();

int main()
{
    A a;
    B b;

    A *ap = &a;
    if (dynamic_cast<B *>(ap))
        cout << "OK1" << endl;
    else
        cout << "FAIL" << endl;
    if (static_cast<B *>(ap))
        cout << "OK2" << endl;
    else
        cout << "FAIL" << endl;

    ap = &b;
    if (dynamic_cast<B *>(ap))
        cout << "OK3" << endl;
    else
        cout << "FAIL" << endl;
    if (static_cast<B *>(ap))
        cout << "OK4" << endl;
    else
        cout << "FAIL" << endl;
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

The 4th line is (1分)

4-12

4-13 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
    static int m;
    int n;
public:
    A(int m,int n){
        this->m = m;
        this->n = n;
    }
    void print(){
        cout << m << "----" << n << endl;
    }
};
int A::m;

int main(){
    A a1(3,4);
    A a2(5,6);
    a1.print();
    a2.print();
}
```

The 1st line is (2分)

The 2nd line is (1分)

4-13

4-14 The output of the code below is:

```
#include <iostream>
using namespace std;

class A{
public:
    A(){ cout << "A()" << endl;}
    A(const A&){ cout << "A(const A&)" << endl;}
    A &operator=(const A&){
        cout << "operator=(const A&)" << endl;
        return *this;
    }
};

int main()
{
    A a1,a2;
    a2 = a1;
    A a3 = a2;
}
```

The 1st line is (1分)

The 2nd line is (1分)

The 3rd line is (1分)

The 4th line is (1分)

4-14

4-15 The output of the code below is:

```
#include <iostream>

struct Base
{
    virtual ~Base()
    {
        std::cout << "Destructing Base" << std::endl;
    }
    virtual void f()
    {
        std::cout << "I'm in Base" << std::endl;
    }
};

struct Derived : public Base
{
    ~Derived()
    {
        std::cout << "Destructing Derived" << std::endl;
    }
    void f()
    {
        std::cout << "I'm in Derived" << std::endl;
    }
};

int main()
{
    Base *p = new Derived();
    (*p).f();
    p->f();
    delete p;
}
```

The 1st line: (1分)

The 2nd line: (1分)

The 3rd line: (1分)

The 4th line: (1分)

4-15

2018-2019 OOP 期末考试

✖ True-or-False 10

A. Multiple-Choice - 1 10

📄 Fill-in-Blank 9

📄 Fill-in-Blank - P 3

4-1 The output of the code below is:

```
#include<iostream>
using namespace std;
class MyClass {
public:
    MyClass(int x): val(x) {}
    void Print() const {cout << 1 << val;}
    void Print() {cout << 2 << val;}
private:
    int val;
};
int main() {
    const MyClass obj1(10);
    MyClass obj2(20);
    obj1.Print();
    obj2.Print();
    return 0;
}
```

Author: 翁恺

Organization: 浙江大学

110220 (3分)

4-1 Accepted (3 point(s))

4-2 The output of the code below is:

```
#include<iostream>
using namespace std;
class AA {
public:
    AA() { cout << 1; }
    ~AA() { cout << 2; }
};
class BB: public AA {
    AA aa;
public:
    BB() { cout << 3; }
    ~BB() { cout << 4; }
};
int main() {
    BB bb;
    return 0;
}
```

Author: 翁恺

Organization: 浙江大学

113422 (3分)

4-2 Accepted (3 point(s))

4-3 The output of the code below is:

```
#include <iostream>
using namespace std;

class A {
public:
    A() { cout << 1; }
} a;

int main()
{
    cout << 2;
    A a;

    return 0;
}
```

Author: 翁恺

Organization: 浙江大学

121 (3分)

4-3 Accepted (3 point(s))

4-4 write the output of the code below.

Author: hulanqing
Organization: 浙江大学

```
#include<iostream>
using namespace std;

class INCREMENT
{
public:
    INCREMENT( int v = 0, int i = 1 );
    void addIncrement()
    {
        v += increment;
    }
    void print() const;
    int get() const
    {
        return v;
    }
private:
    int v;
    const int increment;
};

INCREMENT::INCREMENT( int v, int i ) : v( v ), increment( i )
{
}

void INCREMENT::print() const
{
    cout << v << endl;
}

int main()
{
    INCREMENT value( 1, 2);
    value.print();

    for ( int j = 1; j <= 2; j++ )
    {
        value.addIncrement();
        value.print();
    }
    return 0;
}
```

One for each line:

line 1: (1分) line 2: (1分)
line 3: (1分)

4-4 Accepted (3 point(s))



4-5 write the output of the code below.

Author: hulanqing
Organization: 浙江大学

```
#include<iostream>
using namespace std;
class TEST
{
    int num;
public:
    TEST( int num=0);
    void increment( ) ;
    ~TEST( );
};
TEST::TEST(int num) : num(num)
{
    cout << num << endl;
}
void TEST::increment()
{
    num++;
}
TEST::~~TEST( )
{
    cout << num << endl;
}
int main( )
{
    TEST array[2];
    array[0].increment();
    array[1].increment();
    return 0;
}
```

One for each line:

line 1:	0	(1分)
line 2:	0	(1分)
line 3:	1	(1分)
line 4:	1	(1分)

4-5 Accepted (4 point(s))

4-6 The output of the code below is:

Author: 翁恺
Organization: 浙江大学

```
#include <iostream>
using namespace std;

class MyClass {
public:
    MyClass() {
        ++count;
    }
    ~MyClass() {
        --count;
    }
    static int getCount() {
        return count;
    }
private:
    static int count;
};
int MyClass::count = 0;
int main() {
    MyClass obj;
    cout << obj.getCount();
    MyClass obj2;
    cout << MyClass::getCount();
    cout << obj2.getCount();
    return 0;
}
```

122 (3分)

4-6 Accepted (3 point(s))

4-7 write the output of the code below.

 Author: hulanqing
Organization: 浙江大学

```
#include<iostream>
using namespace std;

enum NOTE { middleC, Csharp, Cflat };
class Instrument {
public:
    virtual void play(NOTE) const = 0;
    virtual char* what() const = 0;
    virtual void adjust(int) = 0;
};

class Wind : public Instrument {
public:
    void play(NOTE) const {
        cout << 1 << endl;
    }
    char* what() const { return "Wind"; }
    void adjust(int) {}
};

class Percussion : public Instrument {
public:
    void play(NOTE) const {
        cout << 2 << endl;
    }
    char* what() const { return "Percussion"; }
    void adjust(int) {}
};

class Stringed : public Instrument {
public:
    void play(NOTE) const {
        cout << 3 << endl;
    }
    char* what() const { return "Stringed"; }
```

```

void adjust(int) {}
};

class Brass : public Wind {
public:
    void play(NOTE) const {
        cout << 11 << endl;
    }
    char* what() const { return "Brass"; }
};

class Woodwind : public Wind {
public:
    void play(NOTE) const {
        cout << 12 << endl;
    }
    char* what() const { return "Woodwind"; }
};

void tune(Instrument& i) {
    i.play(middleC);
}

void f(Instrument& i) { i.adjust(1); }

int main() {
    Wind flute;
    Percussion drum;
    Stringed violin;
    Brass flugelhorn;
    Woodwind recorder;
    tune(flute);
    tune(drum);
    tune(violin);
    tune(flugelhorn);
    tune(recorder);
    f(flugelhorn);
    return 0;
}

```

One for each line:

line 1:	1	(1分)
line 2:	2	(1分)
line 3:	3	(1分)
line 4:	11	(1分)
line 5:	12	(1分)

4-7 Accepted (5 point(s))

4-8 write the output of the code below.

Author: hulanqing
Organization: 浙江大学

```
#include<iostream>
#include<string>
using namespace std;

class Pet {
public:
    virtual string speak() const { return "pet!"; }
};

class Dog : public Pet {
public:
    string speak() const { return "dog!"; }
};

int main() {
    Dog ralph;
    Pet* p1 = &ralph;
    Pet& p2 = ralph;
    Pet p3;
    cout << p1->speak() <<endl;
    cout << p2.speak() << endl;
    cout << p3.speak() << endl;
    return 0;
}
```

dog! (1分)

dog! (1分)

pet! (1分)

4-8 Accepted (3 point(s))

4-9 The output of the code below is:

Author: 翁恺
Organization: 浙江大学

```
#include <iostream>
using namespace std;

class A {
    int i;
public:
    A() : i(0) {}
    ~A() { cout << get(); }
    void set(int i) { this->i = i; }
    int get() { return i; }
};

int main()
{
    A* p = new A[2];
    delete p;
    return 0;
}
```

0 (3分)

4-9 Accepted (3 point(s))

I. 判断题

共 10 小题，共计 10 分

II. 单选题

共 15 小题，共计 30 分

III. 填空题

共 10 小题，共计 32 分

IV. 程序填空题

共 4 小题，共计 38 分

剩余时间: 19:14:41

判断题

单选题

填空题

程序填空题

1-1 Order of initialization in the initial list is the order of their declaration in the list. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

1-2 An abstract class is a class with at least one pure virtual function. (1分)

☒ T ☐ F

作者: 翁恺
单位: 浙江大学

1-3 The operator `:::` can not be overloaded. (1分)

☒ T ☐ F

作者: 翁恺
单位: 浙江大学

1-4 Dynamic binding is used as default binding method in C++. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

1-5 One class can have more than one super classes. (1分)

☒ T ☐ F

作者: 翁恺
单位: 浙江大学

1-6 The index of an array of pointers to objects starts from 1. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

1-7 In C++, inheritance allows a derived class to directly access all of the functions and data of its base class. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

1-8 Destructors can not be overloaded. (1分)

☒ T ☐ F

作者: 翁恺
单位: 浙江大学

1-9 Every C++ compiler guarantees that `sizeof(int)` is less than `sizeof(long)`. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

1-10 A program is a bunch of objects telling each other how to do by sending messages. (1分)

☐ T ☒ F

作者: 翁恺
单位: 浙江大学

判断题

1	2	3	4	5
6	7	8	9	10

单选题

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15

填空题

1	2	3	4	5
6	7	8	9	10

程序填空题

1	2	3	4
---	---	---	---

判断题 单选题 填空题 程序填空题

2-1 Who can access a private member of a class? (2分)

- ☐ A. Only member functions of that class.
- ☒ B. Only member functions of that class and friend functions or member functions of friend classes
- ☐ C. Only member functions of that class and derived classes
- ☐ D. None of the others

作者: 翁恺
单位: 浙江大学

2-2 Resolver `::` is used to: (2分)

- ☐ A. Define a member function outside class declaration
- ☐ B. Access a member of a namespace
- ☐ C. Access a static member of a class
- ☒ D. All of the others

作者: 翁恺
单位: 浙江大学

2-3 Given:

```
class ResId {
public:
    ResId(int Id);
};
```

作者: 翁恺
单位: 浙江大学

The expression `ResId res = 5` means: (2分)

- ☐ A. A temporary object of class ResId will be created by 5
- ☒ B. An object of class ResId will be created by 5
- ☐ C. An object of class ResId will be created by a temporary object
- ☐ D. The object res will be assigned with the temporary object that was created by 5

2-4 Given:

```
void f(int i) { cout << "Func1" << endl; }
template<class T>
void f(T t) { cout << "Func2" << endl; }
main() {
    f(2);
}
```

作者: 翁恺
单位: 浙江大学

The result is : (2分)

- ☒ A. Func1
- ☐ B. Func2
- ☐ C. *nothing*
- ☐ D. undetermined

2-5 Given:

```
class A {
    A() {};
    virtual f() {};
    int i;
};
```

作者: 翁恺
单位: 浙江大学

which statement is NOT true: (2分)

- ☐ A. i is private

判断题

1	2	3	4	5
6	7	8	9	10

单选题

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15

填空题

1	2	3	4	5
6	7	8	9	10

程序填空题

1	2	3	4
---	---	---	---

- ☐ B. f() is an inline function
- ☐ C. i is a member of class A
- ☒ D. sizeof(A) == sizeof(int)

2-6 Given:

```
class A {
    A() {}
    virtual f() = 0;
    int i;
};
```

作者: 翁恺
单位: 浙江大学

which statement below is **NOT** true: (2分)

- ☐ A. i is private
- ☐ B. Objects of class A can not be created
- ☐ C. i is a member of class A
- ☒ D. sizeof(A) == sizeof(int)

2-7 Given:

```
class X {
    int i;
    virtual void f() {};
};
```

作者: 翁恺
单位: 浙江大学

If sizeof(int *) == sizeof(int) == 4, then sizeof(X) == ? (2分)

- ☐ A. 4
- ☐ B. 6
- ☒ C. 8
- ☐ D. Undetermined

2-8 Which one below is copy constructor of class X?(2分)

- ☐ A. X()
- ☒ B. X(const X& x)
- ☐ C. X(const X& x, int i)
- ☐ D. X(const X* x)

作者: 翁恺
单位: 浙江大学

2-9 Given:

```
template < class T >
void swap( T& x, T& y ) {
    T temp = x;
    x = y;
    y = temp;
}
int i,j;
float f,m;
```

作者: 翁恺
单位: 浙江大学

Which statement is incorrect? (2分)

- ☐ A. swap(i,j);
- ☐ B. swap(j,i);
- ☐ C. swap(f,m)
- ☒ D. swap(i,f);

2-10 For the code below:

```
class A {  
    static int i;  
    //...  
};
```

作者: 翁恺

单位: 浙江大学

Which statement is NOT true?(2分)

- ☒ A. All objects of class A reserve a space for i
- ☐ B. All objects of class A share the space of i
- ☐ C. i is a member variable of class A
- ☐ D. i is allocated in global data space

2-11 Which one is the characteristic of abstract class? (2分)

- ☐ A. May have virtual functions
- ☐ B. May have constructors overloaded
- ☐ C. May have friend function
- ☒ D. Can not make instance of this class

作者: 翁恺

单位: 浙江大学

2-12 In C++ language , function prototype doesn' t identify (2分)

作者: 翁恺

单位: 浙江大学

PTA | 程序设计类实验辅助教学平台

- ☐ A. The return type of the function.
- ☐ B. The number of arguments of the function.
- ☒ C. The functionality of the function
- ☐ D. The type of arguments of the function.

2-13 In a C++ program, objects communicate each other by (2分)

- ☐ A. inheritance
- ☐ B. encapsulation
- ☒ C. calling member functions
- ☐ D. function overloading

作者: 翁恺

单位: 浙江大学

2-14 For an arbitrary class , the number of destructors can' t be greater than (2分)

- ☐ A. 0
- ☐ B. 2
- ☒ C. 1
- ☐ D. 3

作者: 翁恺

单位: 浙江大学

2-15 Suppose a class is defined without any keywords such as public, private and protected , all members default to (2分)

- ☐ A. public
- ☐ B. protected
- ☒ C. private
- ☐ D. static

作者: 翁恺

单位: 浙江大学

保存

浙江大学2013-2014学年春夏学期

《面向对象程序设计》课程期末考试试卷

课程号：_____，开课学院：_____

考试试卷：A卷√ B卷（请在选定项上打√）

考试形式：闭√、开卷（请在选定项上打√），允许带_____入场

考试日期：2014年06月26日，考试时间：120分钟

诚信考试，沉着应考，杜绝违纪。

考生姓名：_____学号：_____所属院系：_____

题序 一 二 三 四 五 六 七 八 总分

得分

评卷人

1. Write the output of the code below (30%)

（每题3分，主要部分对的酌情扣1-2分）

1)

```
class A{
public:
    A& operator=(const A& r)
    {
        cout << "In A::operator=().";
    }
};
class B{
public:
    B& operator=(const B& r)
    {
        cout << "In B::operator=().";
    }
};
class C{
private:
    A a;
    B b;
    int c;
public:
};

void main()
{
    C m,n;
    m = n;
}
```

答案:

In A::operator=().

In B::operator=().

2) #include <iostream>
using namespace std;

```
class counter{
private:
    int value;
public:
    counter():value(0)
    {}
    counter& operator++();
    int operator++(int);
    void reset()
    {
        value = 0;
    }
    operator int() const
    {
        return value;
    }
};

counter& counter::operator++()
{
    if (5 == value)
        value = 0;
    else
        value += 1;

    return *this;
}

int counter::operator++(int)
{
}
```

```

    int t = value;
    if (5 == value)
        value = 0;
    else
        value += 1;

    return t;
}

void main()
{
    counter a;
    while (++a)
        cout << "*****\n";
    cout << 0+a << endl;
    while (a++)
        cout << "*****\n";
    cout << 0+a << endl;
}
答案:
*****
*****
*****
*****
*****
0
1

3)
class Obj {
    char c;
public:
    Obj(char cc){
        c = cc;
        cout << "Obj::Obj(char cc) for " << c << endl;
    }
    ~Obj() {
        cout << "Obj::~Obj() for " << c << endl;
    }
};
void f() { static Obj b('b'); }
void g() { Obj c('c'); }

Obj a('a');
int main()
{
    cout << "inside main()" << endl;
    f();
    g();
    f();
    g();
    cout << "leaving main()" << endl;
    return 0;
}
答案:
Obj::Obj(char cc) for a
inside main()
Obj::Obj(char cc) for b
Obj::Obj(char cc) for c
Obj::~Obj() for c
Obj::Obj(char cc) for c
Obj::~Obj() for c
leaving main()
Obj::~Obj() for b
Obj::~Obj() for a

4)
template <class T>
void print(const T &val){ cout << val; }

template <>
void print(const double &f_val){ cout << (int)f_val << endl ;}

void main()
{
    print("Today's temperature: ");
    print(26.3);
}
答案:
Today's temperature: 26

5)

```

```

int f(int a)
{
    return ++a;
}
int g(int& a)
{
    return ++a;
}

void main()
{
    int m = 0, n = 0;
    m += f(g(m));
    n += f(f(n));
    cout << "m=" << m << endl;
    cout << "n=" << n << endl;
}

```

答案:

m=3

n=2

```

6)
template <typename T>
class FF{
    T a1,a2,a3;
public:
    FF(T b1, T b2, T b3):a1(b1),a2(b2),a3(b3)
    {}
    T Sum() const
    {
        return a1 + a2 + a3;
    }
};
void main()
{
    FF<int> x(2,3,4),y(-2,-3,-4);
    cout << x.Sum() << "\t" << y.Sum() << endl;
}

```

答案:

9 -9

```

7)
class A{
public:
    A() { cout << "A::A() called.\n"; }
    virtual ~A() { cout << "A::~A() called.\n"; }
};
class B:public A
{
public:
    B(int i)
    {
        cout << "B::B() called.\n";
        buf = new char[i];
    }
    virtual ~B()
    {
        delete[] buf;
        cout << "B::~B() called.\n";
    }
private:
    char *buf;
};

void fun(A* p)
{
    delete p;
}

```

```

void main()
{
    A *p = new B(15);
    fun(p);
}

```

答案:

A::A() called.

B::B() called.

B::~B() called.

A::~A() called.

8)

class A

```

{
public:
    A() { cout << "A( )" << endl;}
    ~A() {cout << "~A()" << endl;}
};
class B : public A
{
public:
    B() { cout << "B( )" << endl;}
    ~B() {cout << "~B()" << endl;}

};
void main()
{
    A *ap = new B[2];
    delete ap;
}
答案:
A( )
B( )
A( )
B( )
~A()
9)
class A
{
public:
    virtual ~A(){}
};
class B : public A
{
};
void main()
{
    B *bp;
    B b;
    A a1;
    A &a2 = b;
    try{
        bp = dynamic_cast<B *>(&a1);
        if (bp)
            cout << "Dynamic_cast (1) OK!"<<endl;
        else
            cout << "Dynamic_cast (1) Fail!"<<endl;
        bp = dynamic_cast<B *>(&a2);
        if (bp)
            cout << "Dynamic_cast (2) OK!"<<endl;
        else
            cout << "Dynamic_cast (2) Fail!"<<endl;
        B &b1 = dynamic_cast<B &>(a1);
        cout << "Dynamic_cast (3) OK!" <<endl;
    }
    catch(...){
        cout << "Dynamic_cast (3) Fail!"<<endl;
    }
}
答案:
Dynamic_cast (1) Fail!
Dynamic_cast (2) OK!
Dynamic_cast (3) Fail!
10)
class A
{
public:
    A(){cout << "A()" << endl;}
    A(const A&){cout << "A(const A&)"<<endl;}
    ~A(){cout << "~A()" << endl;}
    A &operator =(const A&){
        cout << "operator="<<endl;
        return *this;
    }
};
void main()
{
    A a1,a2;
    a2 = a1;
    A a3 = a1;
}
答案:
A()
A()

```

```
operator=
A(const A&)
~A()
~A()
~A()
```

2. Please correct the following programs (point out the errors and correct them) (15%) (每题3分)

```
1)
#include <iostream.h>
class A
{
    int m;
    static int k;
public:
    A():m(1111){}
    static int GetM()const{
        return m;
    }
    static int GetK()const{
        return k;
    }
};
int A::k = 555;
void main()
{
    A a;
    cout << a.GetM()<<endl;
    cout << a.GetK()<<endl;
}
```

答案：

```
class A
{
    int m;
    static int k;
public:
    A():m(1111){}
    static int GetM()const{ //去掉static
        return m;
    }
    static int GetK()const{ //去掉const
        return k;
    }
};
int A::k = 555;
void main()
{
    A a;
    cout << a.GetM()<<endl;
    cout << a.GetK()<<endl;
}
```

```
2)
#include <typeinfo.h>
#include <iostream.h>
class A
{
    int m_x;
public:
};
class B : public A
{
    int m_y;
public:
    B(int x = 0, int y = 0){ m_x = x; m_y = y; }
};
void main()
{
    A *ap=new B;
    cout<<typeid(*ap).name()<<endl;
}
```

答案：

```
#include <typeinfo.h>
#include <iostream.h>
class A
{
    int m_x;
protected:
```

```

    int m_x;          //m_x变量声明为protected
public:
    virtual ~A(){}      //增加虚析构函数
};
class B : public A
{
    int m_y;
public:
    B(int x = 0,int y = 0){ m_x = x; m_y = y; }
};
void main()
{
    A *ap=new B;
    std::cout<<typeid(*ap).name()<<std::endl;
    delete ap;        //增加delete ap
}

3)
#include <iostream.h>
class Rational
{
public:
    Rational(int numerator = 0,int denominator = 1){
        n = numerator,d = denominator;}
private:
    int n, d;    // numerator and denominator

    friend const Rational &operator*(const Rational& lhs, const Rational& rhs);
    friend bool operator==(const Rational& lhs, const Rational& rhs);
};

const Rational &operator*(const Rational& lhs, const Rational& rhs)
{
    static Rational result;
    //multiply lhs by rhs and put the product inside result
    result.n = lhs.n * rhs.n;
    result.d = lhs.d * rhs.d;
    return result;
}
bool operator==(const Rational &lhs,const Rational &rhs)
{
    return lhs.n * rhs.d == rhs.n * lhs.d;
}
void main(){
    Rational a(1,2), b(3,5), c(2,1), d(1,7);
    if ((a * b) == (c * d)) {
        cout << "Equal" << endl;
    }
    else {
        cout << "Unequal" << endl;
    }
}

答案：
#include <iostream.h>
class Rational
{
public:
    Rational(int numerator = 0,int denominator = 1){
        n = numerator,d = denominator;}
private:
    int n, d;    // numerator and denominator

    friend const Rational &operator*(const Rational& lhs, const Rational& rhs);    //去掉&
    friend bool operator==(const Rational& lhs, const Rational& rhs);
};

const Rational &operator*(const Rational& lhs, const Rational& rhs)
{    //去掉&
    static Rational result;    //去掉static
    //multiply lhs by rhs and put the product inside result
    result.n = lhs.n * rhs.n;
    result.d = lhs.d * rhs.d;
    return result;
}
bool operator==(const Rational &lhs,const Rational &rhs)
{
    return lhs.n * rhs.d == rhs.n * lhs.d;
}
void main(){
    Rational a(1,2), b(3,5), c(2,1), d(1,7);

```

```

    if ((a * b) == (c * d)) {
        cout << "Equal" << endl;
    }
    else {
        cout << "Unequal" << endl;
    }
}

4)
class B {
public:
    virtual void f(){}
};
class D: public B {
public:
    virtual void f() const{}
};

void main()
{
    const B *bp = new D;
    bp->f();
}

答案:
class B {
public:
    virtual void f() const{}    //加上const
};
class D: public B {
public:
    virtual void f() const{}
};

void main()
{
    const B *bp = new D;
    bp->f();
    delete bp;    //增加delete bp
}

5)
class A
{
public:
    static int f1() const
    {
        return m_i;
    }
    static int f2() const
    {
        return m_s;
    }
    static int f3() const
    {
        return ++m_i;
    }
private:
    int m_i;
    static int m_s;
};
int A::m_s = 0
答案:
class A
{
public:
    static int f1() const    //去掉static
    {
        return m_i;
    }
    static int f2() const    //去掉static
    {
        return m_s;
    }
    static int f3() const    //去掉static和const
    {
        return ++m_i;
    }
private:
    int m_i;

```

```

    static int m_s;
};
int A::m_s = 0

3. Fill in the blanks (20%) (每空1分)
#include <iostream>
using namespace std;
class Base{
private:
    int a;
public:
    Base(int a=0) _____
    {}
    virtual const char* what_am_i() const
    {
        return "Base\n";
    }
    _____ ~Base(){}
};
class Derived:public Base{
    char *p;
public:
    Derived(char *p) _____
    {
        _____;
        _____;
    }
    Derived(const Derived& obj) _____
    {
        _____;
        _____;
    }
    _____ what_am_i()_____
    {
        return "Derived\n";
    }
    Derived& operator=(const Derived & rhs)
    {
        if (_____)
            return *this;

        _____;
        _____;
        _____;
        _____;
    }
    void stringIs() const
    {
        cout << p << endl;
    }

    ~Derived()
    {
        _____;
    }
};

void main()
{
    Base *p;
    p = new Derived("hello");
    Derived *q;
    q = _____;
    if (_____)
        q->stringIs();
    cout << p->what_am_i();
    cout << (*p).what_am_i();
    _____;
}

答案:
#include <iostream>
#include <string>
using namespace std;

class Base{
private:
    int a;
public:
    Base(int a=0) __:a(a)_____
    {}

```

```

    virtual const char* what_am_i() const
    {
        return "Base\n";
    }
    __virtual__ ~Base(){}
};
class Derived:public Base{
    char *p;
public:
    Derived(char *p) :Base()_____
    {
        _this->p = new char[strlen(p)+1]_;
        _strcpy(this->p, p)_;
    }
    Derived(const Derived& obj) :Base(obj)_____
    {
        __p = new char[strlen(obj.p)+1]_;
        _strcpy(p, obj.p)_;
    }
    _virtual const char*__ what_am_i()__const__
    {
        return "Derived\n";
    }
    Derived& operator=(const Derived & rhs)
    {
        if (__this == &rhs__)
            return *this;

        _delete[] p_;
        __Base::operator=(rhs)__;
        __p = new char[strlen(rhs.p)+1]__;
        _strcpy(p, rhs.p)__;
        _return *this_____;
    }
    void stringIs() const
    {
        cout << p << endl;
    }
    ~Derived()
    {
        _delete[] p____;
    }
};

void main()
{
    Base *p;
    p = new Derived("hello");
    Derived *q;
    q = _dynamic_cast<Derived*>(p)__;
    if (__q__)
        q->stringIs();
    cout << p->what_am_i();
    cout << (*p).what_am_i();
    _delete p____;
}

```

4. Program Design (35%)

Given declaration of class Person as:

```

class Person {
public:
    Person(char* name);
    Person(const Person& r);
    virtual ~Person() {}
    char* getName() const { return name; }
    virtual void print() const;
    bool operator ==(const Person&) const;
private:
    char* name;
};

```

Your job is to design a simulation program for a clinic, in which there are doctors and patients. To be specific, the tasks you should do are

1. Complete the code for member functions of class Person.
2. Design a class Doctor, which is derived from Person, and represents his/her specializing field as a string, and a registration fee rate.
3. Design a class Patient, which is also derived from Person, and has his/her social security number as a string. Override the print function.
4. Design a class Bill, in which there is one object of Doctor and one object of Patient as members. Design a print function for Bill to print information of all the bills.
5. Write a test program to create at least two doctors, two patients and two bills. Print information of all the bills.

答案：

1. Complete the code for member functions of class Person. (10分)

```

Person::Person(char *nameIn) //五个成员函数每个2分 {
    name = new char[strlen(nameIn)+1];
}

```

```

        strcpy(name, nameIn);
    }
    Person::Person(const Person &other) {
        name = new char[strlen(other.name)+1];
        strcpy(name, other.name);
    }
    void Person::print() const {
        cout << name << endl;
    }
    bool Person::operator==(const Person &other) const {
        return strcmp(this->name, other.name) == 0;
    }
    Person::~Person() {
        delete[] name;
    }
2. Design a class Doctor, which is derived from Person, and represents his/her specializing field as a string, and a registration fee rate
public:
Doctor(char *nameIn, char * fieldIn, int regFreeRateIn)//构造函数1分
    : Person(nameIn), regFreeRate(regFreeRateIn) {
        field = new char[strlen(fieldIn) + 1];
        strcpy(field, fieldIn);
    }
    virtual void print() { //重载print函数2分
        Person::print();
        printf("specializing field: %s\n", field);
        printf("registration free rate: %d\n", regFreeRate);
    }
~Doctor() //析构函数1分
{
    delete[] field;
}
private:
    int regFreeRate; //reg成员变量1分
    char * field; //field成员变量1分
};
3. Design a class Patient, which is also derived from Person, and has his/her social security number as a string. Override the print function
class Patient : public Person { //类名、继承正确1分
public:
    Patient(char *nameIn, char *socialSecurityNumberIn) //构造函数1分
        :Person(nameIn)
    {
        socialSecurityNumber =
            new char[strlen(socialSecurityNumberIn) + 1];
        strcpy(socialSecurityNumber, socialSecurityNumberIn);
    }
    virtual void print() { //重载print函数2分
        Person::print();
        printf("social security Number: %s\n", socialSecurityNumber);
    }
~Patient() //析构函数1分
{
    delete[] socialSecurityNumber;
}
private:
    char *socialSecurityNumber; //成员变量2分
};
4. Design a class Bill, in which there is one object of Doctor and one object of Patient as members. Design a print function for Bill to print
class Bill { //类名2分
public:
    Bill(Patient *patientIn, Doctor *doctorIn) //构造函数2分
        : patient(patientIn),
          doctor(doctorIn) {

    }
    void print() { //print函数2分
        printf("doctor information as below:\n");
        doctor->print();
        printf("patient information as below:\n");
        patient->print();
    }
private: //成员变量2分
    Patient *patient;
    Doctor *doctor;
};
5. Write a test program to create at least two doctors, two patients and two bills. Print information of all the bills. (5分)
#include <stdio.h>
#include <string.h>
int main() {
    Doctor doctor1("Bill", "field 1", 3); //create doctor 1分
    Doctor doctor2("Fredman", "field 2", 4);

```

```

doctor1.print();
doctor2.print();

Patient patient1("Elvis", "0134-443"); //create patient 1分
Patient patient2("Adman", "0244-334");
patient1.print();
patient2.print();

Bill bill1(&patient1, &doctor1); //create bill 1分
Bill bill2(&patient2, &doctor2);
bill1.print(); //打印信息2分
bill2.print();
return 0;
}

```

浙江大学20 14 -20 15 学年 秋冬 学期

《面向对象程序设计》课程期末考试试卷

课程号： 211C0010__， 开课学院：__计算机科学与技术__

考试试卷：A卷√ B卷 (请在选定项上打√)

考试形式：闭√、开卷 (请在选定项上打√)， 允许带__纸笔__入场

考试日期： 2015 年 01 月 20 日, 考试时间： 120 分钟

诚信考试，沉着应考，杜绝违纪。

考生姓名：_____ 学号：_____

所属院系：_____

题序 一 二 三 四 五 六 七 八 总分

得分

评卷人

1. Choices (10%)

Choose the only correct answer from the four choices.

1. Which is the correct way to define a pure virtual function? (C)

- a. virtual void foo();
- b. virtual void foo() {}
- c. virtual void foo() = 0;
- d. void foo(){}

2. Which part of a class can be defined as virtual function? (D)

- a. friend function
- b. constructor function
- c. static member function
- d. destructor function

3. Which function will not be created by compiler if not defined explicitly? (D)

- a. constructor function
- b. destructor function
- c. copy constructor function
- d. inline function

4. Which of the following sentences is not correct (D)

- a. Derived class object can be assigned to Base class object.
- b. Base class object can be assigned to Derived class object.
- c. Derived class object point can be assigned to Base class object point.
- d. Base class object point can be assigned to Derived class object point.

5. What would be printed by the following statements? (C)

```

void main( )
{
    int i = 1;
    double x = 1.111;
    cout << i << " " << x << endl;
    {
        int x = 2;
        double i = 2.222;
        cout << i << " "<< x << endl;
    }
}

```

- A. 1 1.111
- 2 2
- B. 1 1
- 2.222 2
- C. 1 1.111
- 2.222 2
- D. None of the above

2. Write the output of the code below (assuming all the header files are taken care of) (30%, 6% for 2nd question, 4% for the rest)

1) // 缺省构造函数

```

class A {
    int i;
public:
    A() : i(0) {}
    ~A() { cout << get(); }
}

```

```

    void set(int i) { this->i = i; }
    int get() { return i; }
};

int main()
{
    A* p = new A[2];
    delete[] p;
    return 0;
}

Answers:
00
2) // 构造函数与析构函数

class MyClass
{
public:
    MyClass(int id) { cout << "MyClass::Ctor\n"; }
    ~MyClass() { cout << "MyClass::Dtor\n"; }

private:
    int id;
};

class Base
{
public:
    Base(int _id):myclass(_id) { cout << "Base::Ctor\n"; }
    ~Base() { cout << "Base::Dtor\n"; }
    virtual void foo() { cout << "Base::foo\n"; }
private:
    MyClass myclass;
};

class Derived : public Base
{
public:
    Derived(int _id):Base(_id){ cout << "Derived::Ctor\n"; foo(); }
    ~Derived(){ cout << "Derived::Dtor\n"; foo(); }
    virtual void foo() { cout << "Derived::foo\n"; }
private:
};

int main()
{
    Base* p = new Derived(10);
    delete p;
    return 0;
}

Answers:
MyClass::Ctor
Base::Ctor
Derived::Ctor
Derived::foo
Base::Dtor
MyClass::Dtor

3) // 缺省构造函数

class A {
    int i;
public:
    A() : i(0) {}
    virtual void set(int i) { this->i = i; }
    virtual int get() { return i; }
};

class B : public A {
    int i;
public:
    B() : i(10) {}
    virtual void set(int i) { this->i = i; }
};

int main()
{
    B b;
    A* p = &b;
    p->set(30);
}

```

```

        cout << p->get();

        return 0;
    }

Answers:
0

3) // 函数重载

class A {
public:
    void f(int a, int b=10) {cout << a+b; }
    void f(int a, int b=10) const {cout << a-b; }
};

int main()
{
    A a;
    A const * p = &a;
    p->f(25);
    return 0;
}

```

Answers:
15

```

4) // 虚函数

class A {
    int i;
public:
    virtual void set(int ii) { i = ii;}
    virtual int get() { return i; }
};

class B : public A {
    int i;
public:
    virtual void set(int ii) { i = ii;}
    virtual int get() { return i; }
};

int main()
{
    B a;
    B b;
    a.set(10);
    b.set(20);
    A& p = a;
    p.set(30);
    p = b;
    p.set(40);
    cout << a.get();
    return 0;
}

```

Answers:
40

```

5) // 继承

class A{
public:
    int f(){ return 1; }
    virtual int g(){ return 2; }
};

class B : public A{
public:
    int f(){ return 3; }
    virtual int g(){ return 4; }
};

class C : public A{
public:
    virtual int g(){ return 5; }
};

int main(){
    A *pa;
}

```

```

A a;
B b;
C c;

pa = &a;
cout << pa->f() << endl;
cout << pa->g() << endl;
pa = &b;
cout << pa->f() + pa->g() << endl;
pa = &c;
cout << pa->f() << endl;
cout << pa->g() << endl;

```

```

return 0;
}

```

Answers:

```

1
2
5
1
5

```

6) // 继承与虚函数

```

class Base{
public:
    int Bar(char x)
    {
        return (int)(x);
    }
    virtual int Bar(int x)
    {
        return (2*x);
    }
};

```

```

class Derived : public Base{
public:
    int Bar(char x)
    {
        return (int)(-x);
    }
    virtual int Bar(int x)
    {
        return (x/2);
    }
};

```

```

int main()
{
    Derived obj;
    Base* pObj = &obj;

    cout<<pObj->Bar((char)100)<<endl;
    cout<<pObj->Bar(100)<<endl;

    return 0;
}

```

Answers:

```

100
50

```

3. Please correct the following programs (point out the errors and correct them. Please state the reasons if necessary (10%))

1) Please cross out all the lines that will not compile successfully in the main function of the following program

```

#include<iostream>
using namespace std;

class A{
public:
    A(){
        virtual int output() = 0;
    }
private:
    int i;
};

class B : public A{
private:
    int j;
};

```

```

class C{
public:
    int f(int a){ return x*a; }
protected:
    void setX(int a){ x = a; }
    int getX(){ return x; }
private:
    int x;
};

```

```

class D : public C{
private:
    int z;
};

```

```

int main(){
    A* pA = NULL;
    A objA;
    B objB;
    C objC;
    D objD;

    objC.setX(2);
    cout << objC.getX();

    objC.setX(1);
    objC.f(3);

    return 0;
}

```

Answers:

```

int main(){
    A* pA = NULL;
    A objA;
    B objB;
    C objC;
    D objD;

    objC.setX(2);
    cout << objC.getX();

    objC.setX(1);
    objC.f(3);

    return 0;
}

```

2)

```

class A {
virtual void f() { cout << "lala"; }
public:
    void f(int a, int b=10) {cout << a+b; }
    void f(int a, int b=10) const {cout << a-b; }
};

class B : public A {
public:
    void f() { cout << "lili"; }
};

int main()
{
    B a;
    A * p = &a;
    p->f();
    return 0;
}

```

Answers:

A中的f()不能是private的。

3)

```

class A {
    static int k;
public:
    A():k(0) { cout << k; }
};

int main()

```

```

{
    A a;
    return 0;
}

```

答案：

- 1 去掉static。或；
- 2 增加一行全局的：int A::k；并且去掉初始化列表

4. Fill in the blanks (10%)

```

class Employee
{
public:
    Employee( const char * const, const char * const );
    ~Employee();
    const char *getFirstName() const;
    const char *getLastName() const;
    static int getCount();
private:
    char *firstName;
    char *lastName;
    static int count;
};

int Employee::count = 0;
int Employee::getCount()
{
    return count;
}

Employee::Employee( const char * const first, const char * const last )
{
    firstName = new char[ strlen( first ) + 1 ];
    strcpy( firstName, first );

    lastName = new char[ strlen( last ) + 1 ];
    strcpy( lastName, last );

    count++;

    cout << "Employee constructor for " << firstName
        << " " << lastName << " called." << endl;
}
Employee::~~Employee()
{
    cout << "~Employee() called for " << firstName
        << " " << lastName << endl;

    delete [] firstName;
    delete [] lastName;

    count--;
}
const char *Employee::getFirstName() const
{
    return firstName;
}
const char *Employee::getLastName() const
{
    return lastName;
}
int main()
{
    cout << "Number of employees: "
        << Employee::getCount() << endl;

    Employee *e1Ptr = new Employee( "Susan", "Baker" );
    Employee *e2Ptr = new Employee( "Robert", "Jones" );
    cout << "Number of employees: "
        << e1Ptr->getCount();

    cout << "\n\nEmployee 1: "
        << e1Ptr->getFirstName() << " " << e1Ptr->getLastName()
        << "\nEmployee 2: "
        << e2Ptr->getFirstName() << " " << e2Ptr->getLastName() << "\n\n";

    delete e1Ptr;
    e1Ptr = 0;
    delete e2Ptr;
    e2Ptr = 0;
    cout << "Number of employees: "

```

```

        << Employee::getCount() << endl;
        return 0;
    }

    Number of employees: 0
    Employee constructor for Susan Baker called.
    Employee constructor for Robert Jones called.
    Number of employees: 2

    Employee 1: Susan Baker
    Employee 2: Robert Jones

    ~Employee() called for Susan Baker
    ~Employee() called for Robert Jones
    Number of employees: 0

5. Program Design (40%)
Please implement your string class with functions listed below like the string class in <string> and write a main function to test them.

/* you may need use following built-in functions

char * strcpy ( char * destination, const char * source );
size_t strlen ( const char * str );
int strcmp ( const char * str1, const char * str2 );

*/

class MyString{
friend ostream& operator<< (ostream&, MyString&); (4%)
friend istream& operator>> (istream&, MyString&); (4%)
public:
    MyString(const char* str = NULL); (4%)
    MyString(const MyString &other); (4%)
    MyString& operator=(const MyString& other); (4%)
    bool operator==(const MyString&); (4%)
    char& operator[](unsigned int); (4%)
    size_t size();(4%)
    ~MyString(); (4%)
private:
    char *m_data;
};

int main () (4%)

KEY
class MyString{
    friend ostream& operator<< (ostream&, MyString&); (1)
    friend istream& operator>> (istream&, MyString&); (5)
public:
    MyString(const char* str = NULL); (2)
    MyString(const MyString &other); (3)
    MyString& operator=(const MyString& other); (4)
    bool operator==(const MyString&); (6)
    char& operator[](unsigned int); (7)
    ~MyString(); (8)
private:
    char *m_data;
};
Main () (9)

MyString::MyString(const char* str) //1分
{
    if (!str) //1分
        m_data = NULL;
    else {
        int slen = strlen(str);
        m_data = new char[slen + 1]; //以上两行1分
        strcpy_s(m_data, slen+1, str); //1分
    }
}

MyString::MyString(const MyString &other) //1分
{
    if (!other.m_data) //1分
        m_data = NULL;
    else
    {
        int slen = strlen(other.m_data);
        m_data = new char[slen + 1]; //以上两行1分
        strcpy_s(m_data, slen+1, other.m_data); //1分
    }
}

```

```

}

MyString::~MyString() //2分
{
    delete[] m_data; //2分
}

MyString& MyString::operator=(const MyString& other)
{
    if (this != &other) //1分
    {
        delete[] m_data; //0.5分
        if (!other.m_data) m_data = 0; //0.5分
        else //1分
        {
            int slen = strlen(other.m_data);
            m_data = new char[slen + 1];
            strcpy_s(m_data, slen+1, other.m_data);
        }
    }
    return *this; //1分
}

bool MyString::operator==(const MyString &s) //1分
{
    if (strlen(s.m_data) != strlen(m_data))
        return false;
    return strcmp(m_data, s.m_data) ? false : true; //3分
}

char& MyString::operator[](unsigned int e) //1分
{
    if (e >= 0 && e <= strlen(m_data)) //1分
        return m_data[e]; //2分
}

ostream& operator<<(ostream& os, MyString& str) //2分
{
    os << str.m_data; //1分
    return os; //1分
}

istream &operator>>(istream &is, MyString &s) //1分
{
    char temp[255];
    is >> setw(255) >> temp;
    s = temp;
    return is;
}

int main()
{
    MyString str1 = "Aha!";
    MyString str2 = "My friend";
    MyString str3;
    if (!(str1 == str2))
    {
        str3 = str1 + str2;
    }
    cout << str3 << "\n" << str3.size() << endl;
    return 0;
}

```